AI Based Language Translation Compiler

A Python subset compiled to C, where every AI answer is checked by something that can refuse it

Sakrit Kafle 24BCE2952 **Abendra Adhikari 24BCE2952** **Ankur Bhattarai 24BCE2953**

BCSE307P Compiler Design Lab

This is a live demonstration. The slides mark what to show; the application does the talking.

Part 1 The deterministic compiler

Sakrit Kafle - the five stages, what each one produces, and the C that comes out

The problem, and the one rule we hold to

- Python is dynamically typed and garbage collected. C is neither. A translation has to decide types, sizes and lifetimes that Python never writes down.
- So we compile a strict subset: fixed-size arrays, fixed-key dicts, classes as value structs, no allocator.
- The rule the whole project rests on: the output is the specification. A translation is correct only when the compiled C prints the same bytes as CPython.
- That rule is what makes AI usable here. An answer that cannot be checked is not accepted, so a wrong answer becomes a refusal rather than a wrong program.

```
def main() -> int:
    total: int = 0
    for i in range(1, 11):
        total = total + i * i
    print(total)
    return 0
```

```
py_int total = 0LL;
for (py_int i = 1; i < 11; i++)
    total = total + i * i;
printf("%lld\n", (long long)total);
```

DEMO Have the IDE open on this program before you start talking.

Five stages, and exactly where AI is allowed

1. **Lexer**
   tokens, indentation, escapes
2. **Parser**
   recursive descent, Python precedence
3. **Semantic**
   type inference, definite assignment, returns
4. **Codegen**
   rule templates and runtime helpers
5. **out.c**
   self contained; cc out.c -lm -o out

The AI gate

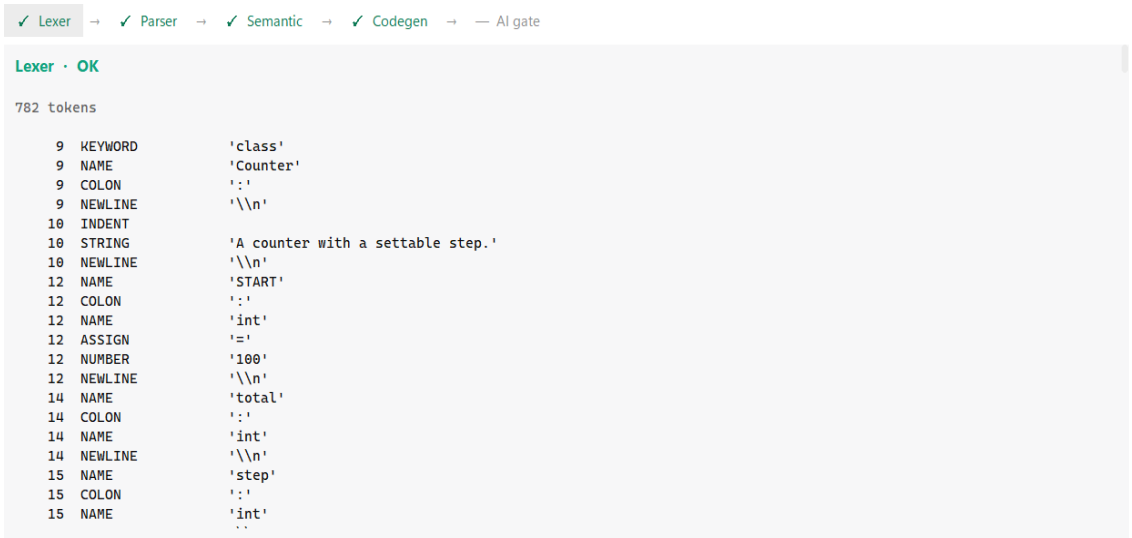
Reached only from Codegen, and only for a call whose name the compiler has no rule for. One C expression, safety checked, compiled, cached.

Stages 1 to 4 contain no AI whatsoever.

DEMO Show the Pipeline tab: the stage chips light up one by one as it compiles.

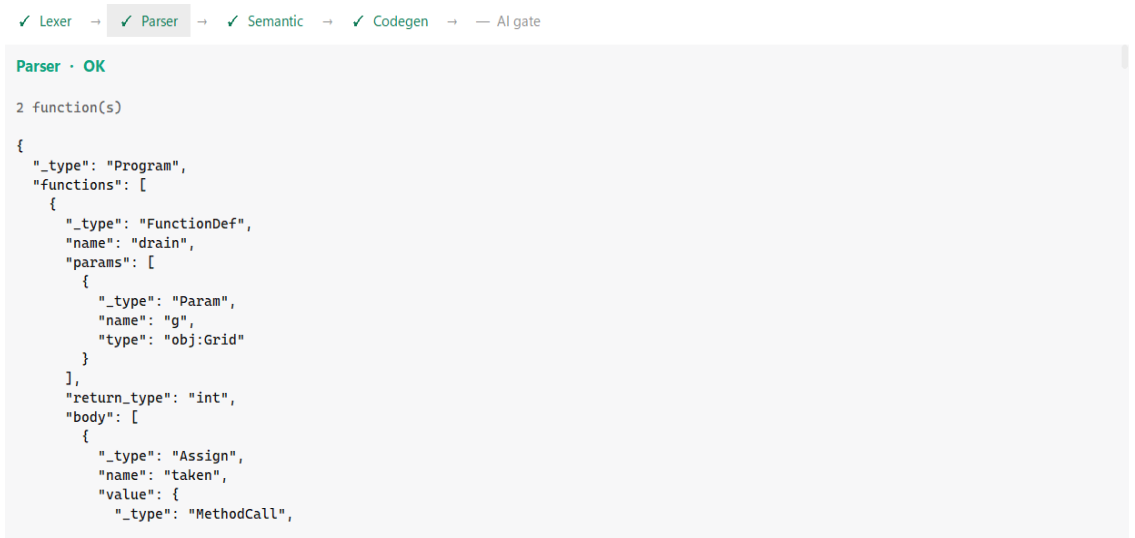
Stages 1 and 2 the front end

Lexer



- Hand written. INDENT and DEDENT come off a column stack - that is how a Python block becomes a C brace.
- Every token carries its line, which is what highlights the failing line later.

Parser



- Recursive descent, one function per precedence level, matching Python's own table.
- What the subset excludes is refused here by name, and the message says what to write instead.

DEMO Click the Lexer chip, scroll the token table, then click Parser and expand a node.

Stage 3 Semantic analysis

✓ Lexer → ✓ Parser → ✓ Semantic → ✓ Codegen → — AI gate

```
Semantic · OK

15 function(s), 29 call site(s)

  def Counter____init__(self: obj:Counter, step: int) -> None

  def Counter__bump(self: obj:Counter) -> None

  def Counter__add(self: obj:Counter, n: int) -> None

  def Counter__value(self: obj:Counter) -> int

  def Counter__from_start(self: obj:Counter) -> int

  def Grid____init__(self: obj:Grid) -> None

  def Grid__put(self: obj:Grid, i: int, v: int) -> None

  def Grid__get(self: obj:Grid, i: int) -> int

  def Grid__total(self: obj:Grid) -> int
       s: int
       i: int
```

- Local type inference as a small dataflow pass. Types only widen along bool < int < float, so it reaches a fixpoint.
- Definite assignment: using a name before every path assigns it is an error here, not a garbage read in C.
- Return coverage: a function declared -> int must return on every path, because C would fall off the end.
- This phase has no AI by design. A wrong type here becomes a silently wrong value in C that no compiler would catch.

DEMO Click Semantic and show the inferred type table.

Inferred types per function, the table codegen uses to hoist C declarations.

Stage 4 Code generation, and the C that comes out

✓ Lexer → ✓ Parser → ✓ Semantic → ✓ Codegen → — AI gate

```
Codegen · OK

29 call site(s), 0 resolved by AI

  Fully deterministic: every construct had a static rule.

Runtime helpers emitted (5):
  py_index
  py_print
  py_print_bool
  py_print_int
  py_print_str
```

- One rule per AST node. Only the runtime helpers a program actually uses are emitted.
- Helpers exist where C disagrees with Python: floor division, modulo of negatives, float formatting, power.
- Every integer literal carries LL. Without it C types the literal int and computes in 32 bits - 10^18 came out as -1486618624.
- The result is one self contained .c file: cc out.c -lm -o out, with no headers of ours to install.

DEMO Press Translate, then F5 to run the generated C. The C runs, not the Python.

The generated C, beside the program it came from.

Part 2 Proof, and the AI that is allowed in

Abendra Adhikari - what byte for byte means, the gate, and whole program repair

What byte for byte actually means

- Not 'the output looks right'. The raw bytes, compared with ==, including the trailing newline and the exit status.
- Captured as bytes, never as text. Python's text mode turns CRLF into LF on both sides, which hides exactly the difference we are looking for.
- The original is run twice before anything else happens. A program that disagrees with itself cannot have a repair verified against it.
- A program that never finishes has no finite output, so it is refused before a single token is spent.

```
target: 115 bytes, exit 0
  your program, CPython      identical
  the rewrite, CPython       identical
  the compiled C, run 1      identical
  the compiled C, run 2      identical
```

ACCEPTED **verified byte for byte**

Four runs, two programs, one answer. Anything less is a refusal.

DEMO In the repair window press 'Run both and compare' - it redoes the comparison live.

The AI gate one expression, and a wall around it

AI-assist policy

- The deterministic core translates the whole language on its own.
- The model is consulted only for a call to a name that is neither defined in your program nor a known builtin.
- Its answer is rejected unless it is a pure math expression over the call's own arguments AND a real C compiler accepts it.
- Verified answers are cached, so a rebuild makes no model call.

Last run: used

What the model translated

Python call on the left, the C expression it became on the right.
a0 and a1 stand for the call's own arguments.

```
hypot(a0, a1)      ->  hypot(a0, a1)   [cache]
sqrt(a0)           ->  sqrt(a0)   [cache] x2 call sites
```

- Fires only for a call the compiler has no rule for, and only for a numeric one.
- The model may return a single C expression. Not a statement, not a function, not an include.
- The answer is checked against a whitelist, then compiled by a real C compiler, then cached by name and arity.
- A cached answer costs nothing on the next run, so the same program twice is one model call, not two.

DEMO Turn AI assist on and translate the program that calls hypotenuse().

The gate's own report: which call sites it resolved, what it returned, and what it cost.

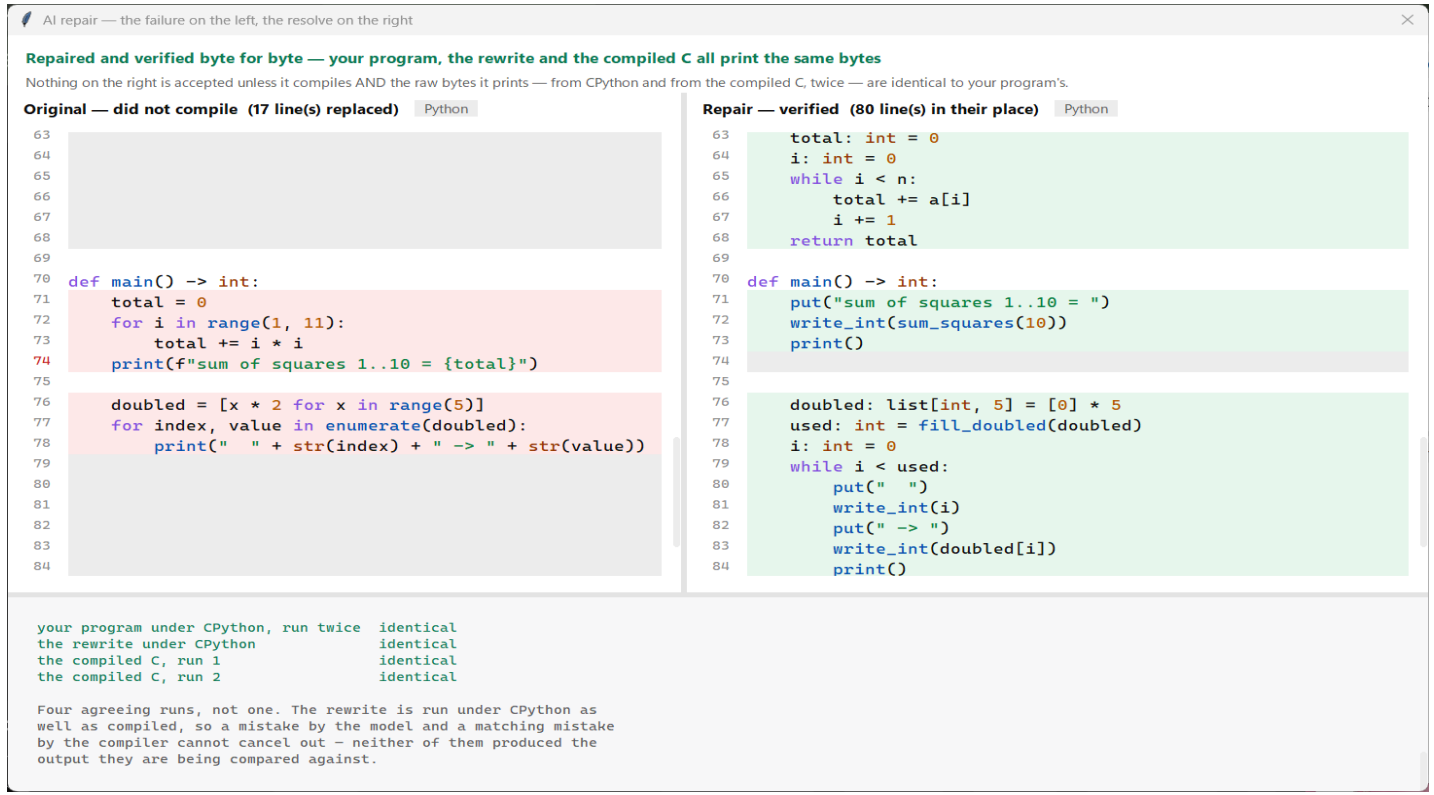
With the gate off, it is a hard error - never a guess

```
$ python -m src.driver tests/programs/ai/01_pythagoras.py --run
tests/programs/ai/01_pythagoras.py: line 6: 'hypot' is not defined in this program and is not one of
 the builtins this compiler translates statically (abs, bool, float, int, len, max, min, pow, put, r
ound). Define it, or enable AI-assist with --ai to have the gate propose and validate a C translatio
n.
```

- The same program with AI assist off does not silently emit something plausible.
- It names the call it cannot translate and stops. The compiler fails closed.
- This is the difference between a tool you can trust and a tool that is usually right.
- It also makes the AI's contribution measurable: turn it off and the program does not compile.

DEMO Toggle AI assist off, translate the same file, show the error naming the call.

Repair when the program itself is outside the subset

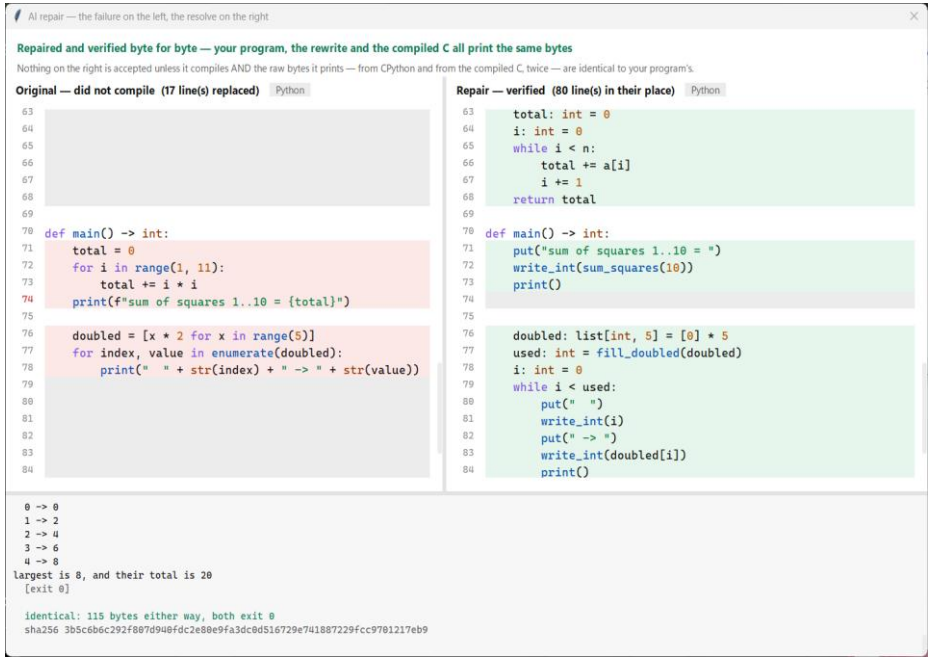


- A different problem from the gate: here the whole program cannot be translated, not one call.
- The model is asked to reimplement it inside the subset - rewrite, not transliterate.
- The rewrite goes through the entire compiler like any other program. Nothing is special cased for it.
- Original on the left, rewrite on the right, and the rewrite is only shown once it has passed.

DEMO Open demos/repair_me.py, translate it, let the repair window open.

What has to be true before a repair is accepted

- **It parses, and declares main()**
  a courtesy check before the compiler sees it
- **The whole compiler accepts it**
  lexer, parser, semantic, codegen
- **It is not a pasted answer**
  printing a constant instead of computing it is refused
- **The C builds**
  with a real C compiler, not a simulation
- **Four runs agree, byte for byte**
  original, rewrite, and the binary twice



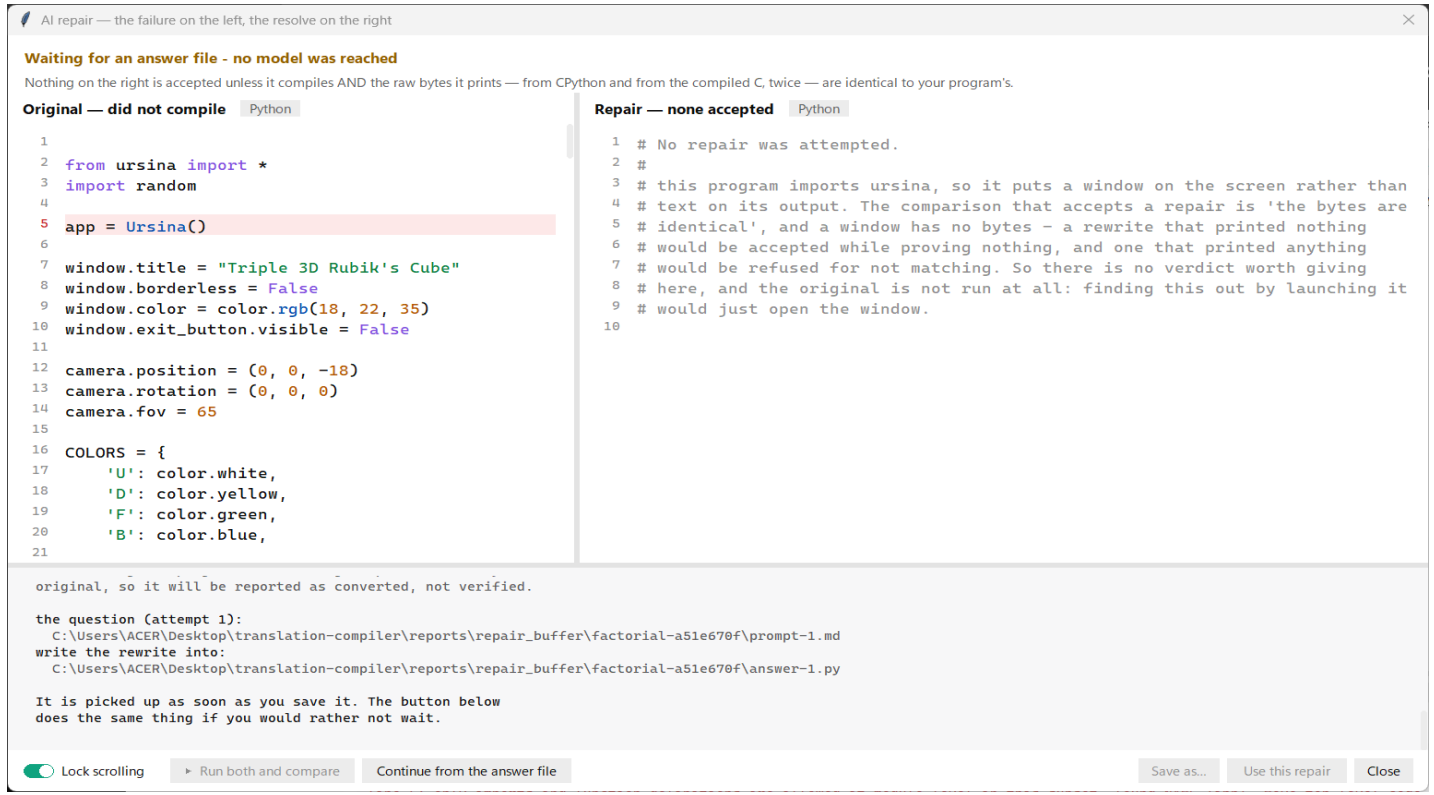
DEMO Point at the verdict line, then press 'Use this repair' to take it into the editor.

Any one of these failing is a refusal, and the original program is left untouched. The third check exists because the prompt shows the model the expected output: a rewrite that prints a figure it never computed is caught and rejected.

Part 3 No key, the last resort, and the limits

Ankur Bhattarai - running without an account, converting what cannot be proved, and what we do not claim

The pipeline does not need an API key



- Nothing in the repair path cares where a rewrite came from. Every check happens after the answer arrives.
- So the question is written to a file, answered anywhere - by a person, or by an assistant in a terminal - and read back.
- The answer is picked up as soon as it is saved, and then goes through every check unchanged.
- An answer written by hand is refused on exactly the same grounds as a model's. The local route is not a privileged one.

DEMO Translate the ursina cube with no credit: the question becomes prompt-1.md on screen.

The handoff: no model was reached, so the compiler writes the question it would have sent.

The last stage convert what cannot be proved



- A program whose behaviour is a window has no output, so byte comparison has nothing to work with.
- Refusing to act left such a program untranslated, which helped nobody.
- So a final stage converts it and reports only what it established: it compiles, it builds and links, it starts and stays up.
- It never reports 'verified'. A test asserts the word never appears alone in this report.

DEMO Let the conversion finish and read the proof list out loud.

CONVERTED, NOT VERIFIED - with what was established, and what was not, side by side.

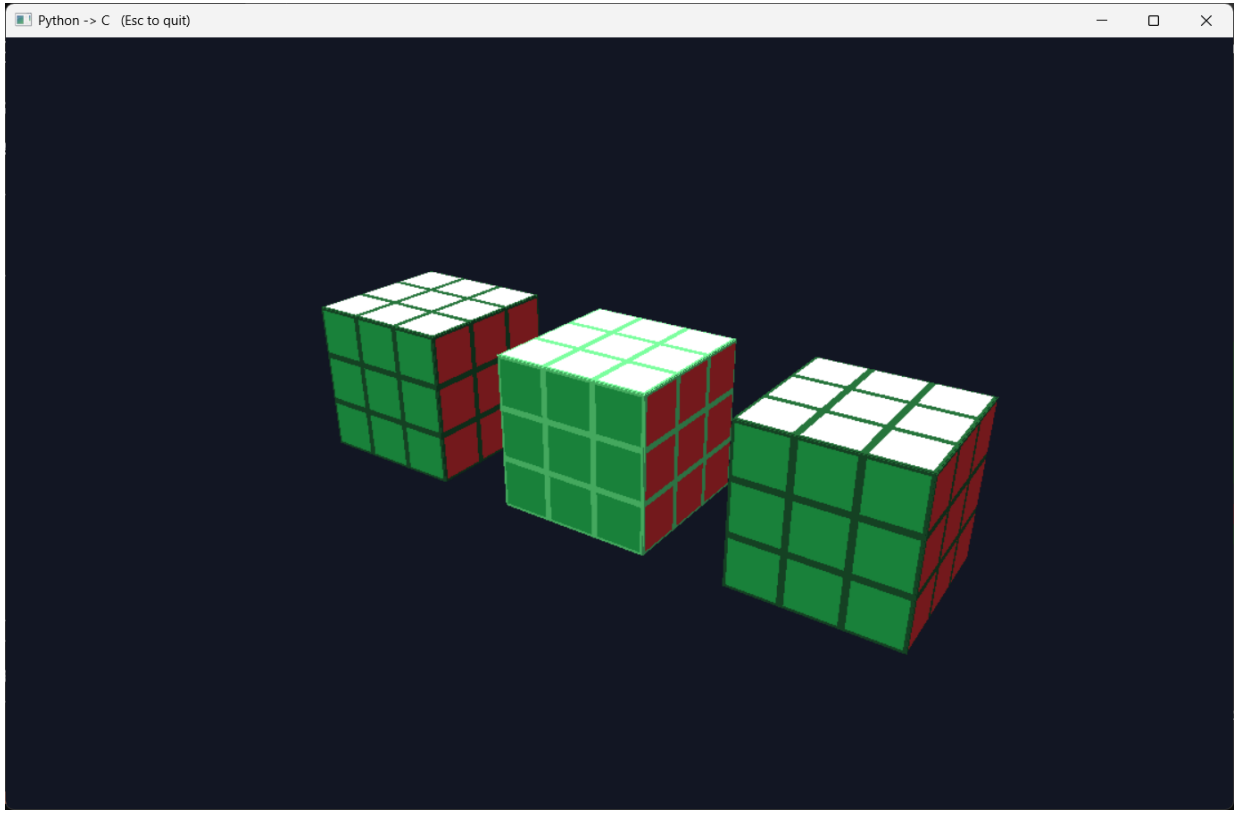
The payoff an ursina program, running as native C

Three Rubik's cubes

The original imports ursina and is refused outright. The conversion draws through gfx, our own window shim, whose API is nothing but numbers.

290,248 bytes

of native executable, no Python underneath




```
1 2 3   select a cube

U D F B R L   turn a face

SHIFT   reverse the turn

S scramble    X reset

SPACE   scramble all three
```

DEMO Run it. Select a cube, turn a face, scramble. Bright grout = selected, green = solved.

Where AI is used, why, and what bounds it

Where	Why it is used	What can refuse it
Gate one C expression	A call the compiler has no rule for. Writing a rule for every maths function is not the point of the project.	Whitelist, then a real C compiler. Cached, so it is asked once.
Simplify tidier C	The emitted C is correct but verbose. A tidier version is a nicety, never a requirement.	Both binaries are built and run; the outputs must match.
Repair whole program	The program is outside the subset entirely, so there is nothing to patch - it has to be rewritten.	The whole compiler, the shortcut check, and four compared runs.
Port last resort	The program's behaviour is a window, so no comparison exists. Without this it stays untranslated.	Compiler, build and start only - and it says so, in those words.

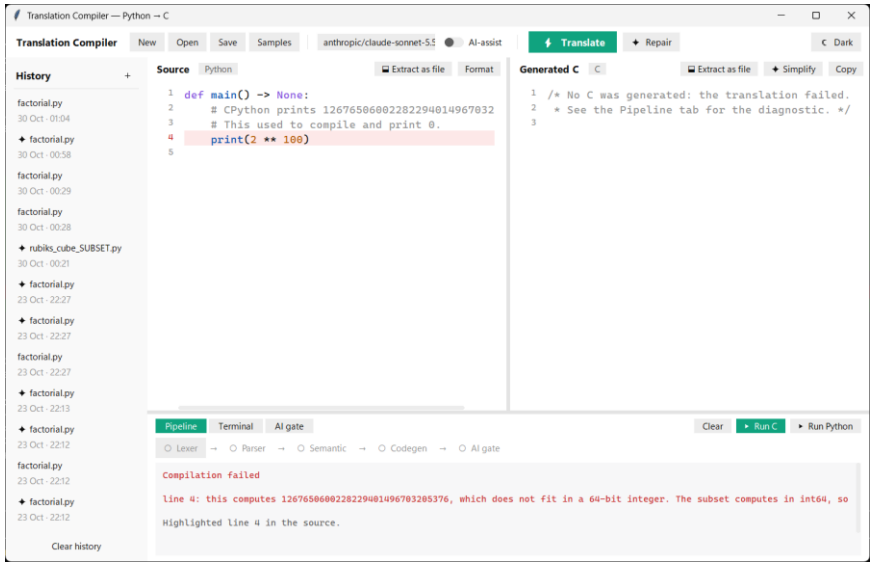
In all four, the AI proposes and something deterministic disposes.

The model is never the thing that decides. It writes text; a whitelist, a C compiler or a byte comparison decides whether that text becomes part of the program.

DEMO Nothing to run here - this is the slide to pause on and answer questions from.

What we do not claim

- A conversion is not a proof. The port stage checks that a program compiles, builds and runs - never that it behaves like the original.
- Integer width. A constant too large for 64 bits is now refused with a diagnostic, but a value only reached at run time still wraps.
- Repair only runs on a program the compiler refused. A program that compiles and prints a wrong answer never reaches it.
- Two refusals are facts about the program, not weaknesses: a program that never finishes has no finite output, and one that disagrees with itself has no fixed output.
- The landing rate is measured, not asserted - and the measurement needs an account, so the number we quote is the last complete run.



*2 ** 100 used to compile and print 0. It is now a diagnostic that names the value - which is also what lets repair act on it.*

Where it stands

724

tests passing,
13 skipped without a provider

13

ordinary Python programs
in the repair corpus

10/12

landed in the last complete
benchmark run

0

AI calls in the
deterministic pipeline

The claim we are willing to defend

Three places in this compiler use a model, and in none of them is the model trusted. Each one is bounded by something that can refuse it - a whitelist, a C compiler, a byte comparison. Where no such bound exists, the result is labelled as a conversion and not as a proof, in those words, on the screen.

DEMO Finish with the test suite running, or the cube still turning on screen.